COVERT — Build Task Breakdown

Confidential Compensation Infrastructure · Zama Season 3 Builder Track

Deadline: July 7, 2026 · Team: 2 people · Deploy target: Sepolia testnet

---

What Covert Is

Covert is a confidential compensation infrastructure protocol built on Ethereum using Zama's FHEVM — Fully Homomorphic Encryption Virtual Machine. It allows organizations to run their entire compensation cycle on-chain — salary distributions, performance bonuses, peer-to-peer recognition — without any participant ever seeing what anyone else earns.

This sounds simple. It is not a small thing.

The Problem It Solves

Every organization that pays people has the same unspoken rule: compensation is private. What you earn is between you and your employer. This norm exists because when people know what colleagues earn, things break — resentment builds, political alliances form, low earners disengage, high earners get poached. The secrecy is not a bug in how organizations work. It is load-bearing infrastructure for team cohesion.

Blockchain, by its nature, destroys this norm entirely. Every transaction on a public chain is visible to everyone. If a company pays its employees in crypto today, any employee — or competitor, or journalist — can look up the wallet addresses, find the transactions, and reconstruct the entire payroll. This is not a hypothetical risk. It is why no serious organization runs payroll on-chain. The transparency that makes blockchain trustworthy is the exact same property that makes it unusable for compensation.

This is not a problem you can solve by moving computation off-chain. If a trusted server computes the salaries and then posts results on-chain, you have reintroduced a single point of trust — and that server can be hacked, subpoenaed, or corrupted. You have not solved the problem. You have hidden it.

Covert solves it at the root. Using Fully Homomorphic Encryption, salary amounts are encrypted before they ever touch the blockchain — and they stay encrypted while the contract processes them, stores them, and distributes them. No one — not the node validators, not the contract deployer, not even Covert itself — can read the encrypted values. The computation happens on ciphertext. The blockchain sees only encrypted numbers it cannot interpret.

How It Actually Works — The Three Roles

Covert has three distinct roles. Each one sees a different slice of the system. This is by design.

The Employer

The employer is the organization running payroll. They are the only party who knows the salary list before it goes on-chain. When they upload payroll, they encrypt each salary amount client-side — in the browser — before the transaction is broadcast. By the time the data hits the blockchain, it is already ciphertext. The smart contract stores it in that encrypted form. The employer can trigger a payroll distribution round, at which point the contract transfers encrypted cUSDT — Zama's confidential USDT — to each employee's wallet. The employer can also fund a peer bonus pool: a separate encrypted budget that employees can distribute among themselves.

The employer can see the total cost of payroll — the aggregate — but even they cannot reconstruct individual salaries from the contract state once submitted. The system is designed so that the moment payroll is uploaded, it belongs to the cryptographic protocol, not to any human administrator.

The Employee

The employee receives their salary as encrypted cUSDT in their wallet. From the outside, their wallet shows they received something. The amount is not visible. To see their own salary, they go through a specific cryptographic handshake called the EIP-712 user-decryption flow. They sign a structured message with their private key — essentially proving to the contract that they are who they claim to be — and the contract releases a decryption key scoped specifically to their data. Their browser decrypts the amount locally. The decrypted number never touches the network.

This is a critical design point: the decryption happens on the employee's device, not on a server. There is no moment where a plaintext salary amount exists anywhere except in the employee's browser session.

Employees also participate in peer bonuses. Each employee receives an encrypted budget — say, 500 cUSDT — to distribute to colleagues who helped them during the month. They can allocate amounts to specific wallet addresses. The smart contract enforces that they cannot exceed their budget without ever revealing how much of that budget has been used or to whom it has been allocated. Colleagues know they received a peer bonus. They do not know the amount, and neither does anyone else except the recipient.

The Auditor

The auditor role exists for compliance and governance. A DAO treasurer, a board member, or a financial regulator might need to verify that the organization is solvent — that it actually disbursed the payroll it claims to have disbursed, and that the numbers add up. Covert gives the auditor access to aggregate statistics: total payroll disbursed this cycle, total bonus pool

allocated, number of recipients. These are computed inside the encrypted domain — the contract adds up encrypted values and returns an encrypted total, which the auditor can decrypt. They never see individual salaries. The sum is provable without the components being visible.

This is the auditor paradox that FHE uniquely solves: you can prove the total without exposing the parts.

Why This Specifically Requires FHE

This is the question every Zama judge will ask, and the answer needs to be airtight.

You cannot build Covert with standard encryption. Standard encryption protects data at rest and in transit — but the moment a smart contract needs to process that data (add up salaries, check budget limits, trigger transfers), it must decrypt it first. And the moment it decrypts on-chain, the data is visible to every node in the network.

You cannot build Covert with zero-knowledge proofs alone. ZK proofs let you prove a statement is true without revealing the underlying data — but they do not allow general computation on private inputs in the way Covert requires. You cannot run a peer bonus budget enforcement check in ZK without extremely complex circuit design that is far outside hackathon scope.

You cannot build Covert with a trusted execution environment (TEE) because TEEs reintroduce hardware trust assumptions and centralization — you are trusting Intel or AMD's chip, not the math.

FHE is the only primitive that allows a smart contract to perform arithmetic on encrypted values — addition, comparison, conditional logic — and produce an encrypted result, all without any party ever holding the decryption key during computation. The key never exists in plaintext on any server at any point. This is not a design choice. It is a mathematical property.

Covert is only possible because FHE exists. That is the sentence the judges need to walk away believing — and it is true.

Who This Is For Right Now

Covert's day-one users are DAOs and crypto-native startups. These organizations already pay contributors in tokens. They already have wallet infrastructure. The pain of on-chain salary transparency is something they live with today — they know their contributor compensation is visible on-chain, and many of them have already had the uncomfortable experience of team members discovering each other's compensation through block explorers.

Traditional companies are a longer-term market. They require off-ramping, fiat bridges, regulatory compliance, and HR system integrations. Covert does not solve all of that. What Covert proves is that the cryptographic primitive — confidential on-chain compensation — works and is usable. Everything else is a product layer built on top.

What Makes It Different From Everything Else Being Submitted

Most submissions to a Zama hackathon encrypt something that did not strictly need to be encrypted on-chain. Covert encrypts something that actively cannot exist on-chain without encryption. The distinction matters enormously. Privacy as a preference is a feature. Privacy as a precondition for the system to function at all — that is architecture.

Compensation secrecy is not a nice-to-have. It is one of the oldest and most universal organizational norms in human employment. Covert is the first implementation of that norm as a cryptographic guarantee rather than a social agreement. You do not have to trust that your employer kept your salary private. The math ensures it.

What We're Building

Covert is a confidential compensation dApp for DAOs and crypto-native teams. It has three roles:

- Employer — uploads encrypted payroll, triggers distributions, manages the compensation cycle
- Employee — receives encrypted salary + peer bonus allocations, decrypts only their own data via EIP-712
- Auditor — views aggregate totals (total disbursed, total bonus pool) without accessing any individual amount

The smart contract layer handles all encrypted computation via FHEVM. The frontend layer handles UX, role routing, and the EIP-712 decryption flow. These two workstreams are cleanly separable — one person owns each.

Smart Contract Dev — Full Task List

Owns everything from contract architecture to deployment. The frontend will call your ABI — coordinate on function signatures early.

#	Task / Deliverable	Owner
C-01	**Project scaffolding** Init Hardhat/Foundry project, install FHEVM libs, configure Sepolia testnet, set up .env and deployment scripts	Smart Contract Dev
C-02	**Define encrypted data types** Define euint64 salary amounts, eaddress employee mappings, encrypted peer bonus budgets using FHEVM types	Smart Contract Dev
C-03	**Role-based access control**	Smart Contract Dev

#	Task / Deliverable	Owner
	Implement three roles: Employer (admin), Employee (recipient), Auditor (aggregate view only). Role assignment and guards	
C-04	**Payroll distribution logic** Employer uploads encrypted salary list. Contract stores encrypted amounts per employee. Trigger distribution function	Smart Contract Dev
C-05	**Peer bonus allocation logic** Each employee gets an encrypted budget. They can allocate portions to colleagues. Budget ceiling enforced without revealing individual amounts	Smart Contract Dev
C-06	**Employee self-decryption flow** Implement EIP-712 user-decryption: employee signs a request, contract returns only their own encrypted amount, frontend decrypts locally	Smart Contract Dev
C-07	**Auditor aggregate view** Auditor role can query total disbursed amount (sum) without accessing any individual salary. Prove solvency without exposure	Smart Contract Dev
C-08	**cUSDT integration** Integrate with Zama's confidential USDT (cUSDT) token. Handle approvals, transfers, balance checks all in encrypted form	Smart Contract Dev
C-09	**Events and error handling** Emit non-sensitive events (PayrollTriggered, BonusAllocated) without leaking amounts. Custom error messages	Smart Contract Dev
C-10	**Unit tests** Full test suite: role guards, encrypted storage correctness, decryption access control, peer bonus ceiling enforcement	Smart Contract Dev
C-11	**Deploy to Sepolia testnet** Deploy contract, seed with test data (3 fake employees, 2 peer bonus rounds). Verify on Etherscan. Document contract address	Smart Contract Dev
C-12	**ABI + contract docs** Export ABI, write NatSpec comments on all functions, produce a one-page contract architecture doc for submission	Smart Contract Dev

Total contract tasks: 12 · Critical path: C-01 → C-02 → C-03 → C-04/C-05 (parallel) → C-06 → C-07 → C-08 → C-09 → C-10 → C-11 → C-12

Frontend Dev — Full Task List

Owns UI, wallet integration, role-gated dashboards, demo seeding, documentation, and the video pitch. Coordinate with Smart Contract Dev on ABI and event structure.

#	Task / Deliverable	Owner
F-01	**Repo setup & tooling** Init Next.js or Vite+React project, install wagmi/viem, ethers.js, Tailwind, connect to Sepolia. Set up folder structure	Frontend Dev
F-02	**Wallet connection**	Frontend Dev

#	Task / Deliverable	Owner
	Integrate RainbowKit or ConnectKit. Detect connected wallet role (Employer / Employee / Auditor) based on contract state	
F-03	**Employer dashboard — payroll upload** UI for employer to input employee wallet addresses + salary amounts. Encrypt values client-side before sending to contract. Batch upload flow	Frontend Dev
F-04	**Employer dashboard — distribution trigger** One-click payroll run button. Show transaction status, gas estimate, confirmation state. Handle errors gracefully	Frontend Dev
F-05	**Employee dashboard — my payslip** Employee sees their encrypted payslip card. Decrypt button triggers EIP-712 signature → reveals their salary amount locally. Animation for reveal moment	Frontend Dev
F-06	**Employee dashboard — peer bonuses** Employee sees their peer bonus budget (encrypted). UI to allocate amounts to colleague addresses. Show remaining budget without revealing allocation history	Frontend Dev
F-07	**Auditor dashboard** Read-only view showing total payroll disbursed, total bonus pool allocated — aggregate numbers only. No individual data visible	Frontend Dev
F-08	**Role-gated routing** Automatically route connected wallet to correct dashboard based on role. Unauthorized access shows clean error state, not broken UI	Frontend Dev
F-09	**Transaction feedback & toasts** Pending / success / failed states for every on-chain action. Loading skeletons for encrypted data fetching	Frontend Dev
F-10	**Responsive UI polish** Mobile-friendly layout. Dark/light mode optional. Typography, spacing, color system consistent. Feels like a real product	Frontend Dev
F-11	**Demo data seeding script** Script to pre-populate testnet with demo state (3 employees, active payroll cycle, 1 peer bonus round) so judges can explore immediately	Frontend Dev
F-12	**README and submission docs** Project README with setup instructions, architecture overview, FHE necessity argument, screenshots. Video pitch script outline	Frontend Dev
F-13	**3-minute video pitch** Record demo video: employer flow → employee decrypt moment → auditor view. Narrate the FHE necessity argument clearly. Edit and export	Frontend Dev

Total frontend tasks: 13 · Critical path: F-01 → F-02 → F-08 → F-03/F-05/F-07 (parallel) → F-04/F-06/F-09 → F-10 → F-11 → F-12 → F-13

Shared Coordination Points

These are not tasks owned by either person — they are handoff moments that both people must agree on before building.

#	Task / Deliverable	Owner
S-01	**Agree on contract ABI** Function names, parameter types, return types for all three roles. Frontend cannot build until this is locked	Both
S-02	**EIP-712 decryption handshake** Smart contract dev implements the signing domain. Frontend dev implements the signature request. Must work end-to-end together	Both
S-03	**Test wallet setup** Three test wallets: Employer, Employee, Auditor. Both devs use same addresses against same deployed contract	Both
S-04	**Demo flow agreement** Agree on the exact 3-minute demo sequence before building. Frontend builds toward it. Contract seeds data for it	Both

Suggested Timeline (12 Days)

Days	Smart Contract Dev	Frontend Dev
Day 1–2	C-01, C-02, C-03 + S-01 ABI agreement	F-01, F-02, F-08 + S-01 ABI agreement
Day 3–5	C-04, C-05, C-06, C-07	F-03, F-05, F-07 (stub against mock ABI)
Day 6–7	C-08, C-09 + S-02 EIP-712 handshake	F-04, F-06 + S-02 EIP-712 handshake
Day 8–9	C-10 (tests), C-11 (deploy) + S-03	F-09, F-10 (polish) + S-03 test wallets
Day 10–11	C-12 (docs, ABI export)	F-11 (seed script), F-12 (README)
Day 12	Review, bug fixes	F-13 (video pitch), final submission

The single most important coordination moment is S-01 on Day 1. Lock the ABI before either person writes a single line of feature code.

Once the ABI is locked, both workstreams are fully independent until the EIP-712 handshake on Day 6–7.